

Experiment No.	4
Experiment Title	Binary Classification using Logistic Regression and Support Vector Machine (SVM) ¹
Student Name	Kavinsoorya K S
Register Number	3122247001030
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

The objective of this experiment is to implement linear and kernel-based binary classifiers, specifically **Logistic Regression** and **Support Vector Machine (SVM)**, to classify emails as spam or ham using the Spambase dataset. Additionally, the experiment aims to analyze the effects of regularization, different kernel choices in SVM, and hyperparameter tuning on overall classification accuracy, precision, recall, and F_1 -score.

2 Problem Statement

Email communication remains vulnerable to unsolicited mail (spam) which degrades network bandwidth and introduces security risks. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$ is a d -dimensional feature vector of word/character frequencies and capital letter run lengths extracted from an email, and $y_i \in \{0, 1\}$ represents the ground-truth label (0 for Ham, 1 for Spam), the objective is to build binary classification models $f(\mathbf{x}) \rightarrow \{0, 1\}$ that maximize classification accuracy while minimizing false positives (classifying ham as spam) and false negatives (failing to identify spam).

3 Dataset Description

The experiment is conducted using the **Spambase Dataset** from the UCI Machine Learning Repository (also accessed via Kaggle). The dataset is composed of 4,601 email records, each containing 57 continuous predictive features. The features represent occurrences of specific keywords, special characters, and length metrics of uppercase letters. The target attribute is a binary class label indicating spam (1) or ham (0). Table 1 summarizes the characteristics of the dataset.

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository / Kaggle
Initial Sample Size	4,601 records
Cleaned Sample Size	4,210 records (391 duplicate rows removed)
Number of Features	57 continuous attributes
Number of Classes	2 (Ham: 0, Spam: 1)
Missing Values	0 (dataset contains no null entries)
Train-Test Split Ratio	80% Training (3,368) / 20% Testing (842)
Training Class Distribution	Ham (0): 2,025 (60.12%), Spam (1): 1,343 (39.88%)

Testing Class Distribution	Ham (0): 506 (60.10%), Spam (1): 336 (39.90%)

The 57 continuous features are categorized as follows:

1. **Word Frequency (features 0–47):** Percentage of words in the email matching specific target words (e.g., `make`, `address`, `all`, `our`, `remove`, `internet`, `free`, `credit`).
2. **Character Frequency (features 48–53):** Percentage of characters in the email matching specific characters (e.g., `;`, `(`, `[`, `!`, `$`, `#`).
3. **Capital Run Length (features 54–56):** Statistics measuring sequences of consecutive capital letters, including the average length (`capital_run_length_average`), longest sequence length (`capital_run_length_longest`), and total number of capital letters (`capital_run_length_total`).

4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the target class distribution, identify features most strongly correlated with spam, and analyze the distribution characteristics of key predictors.

4.1 Target Class Distribution

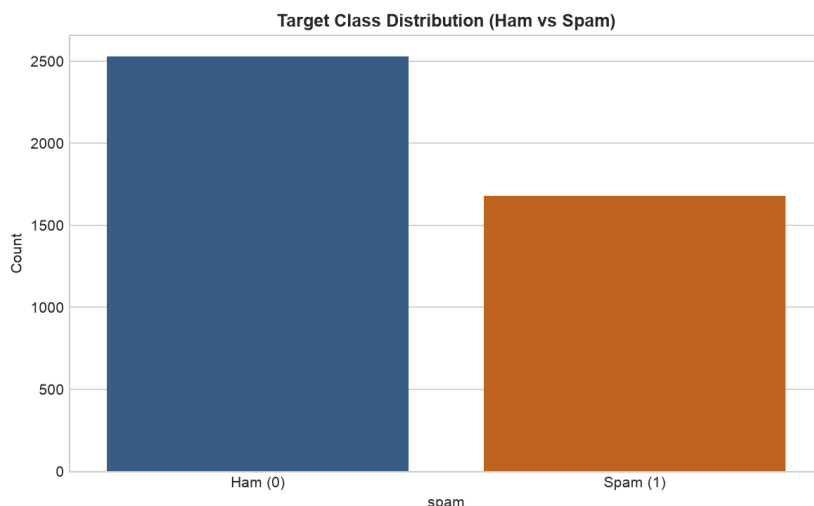


Figure 1: Target Class Distribution showing Ham (0) vs. Spam (1)

Inference: The count plot indicates that the dataset is mildly imbalanced. Ham (legitimate) emails represent approximately 60.1% of the cleaned dataset (2,531 samples), while Spam emails make up about 39.9% (1,679 samples). Standardizing the class proportions in the train-test split via stratified sampling ensures that training and testing partitions mimic this underlying class balance.

4.2 Feature Correlation Analysis

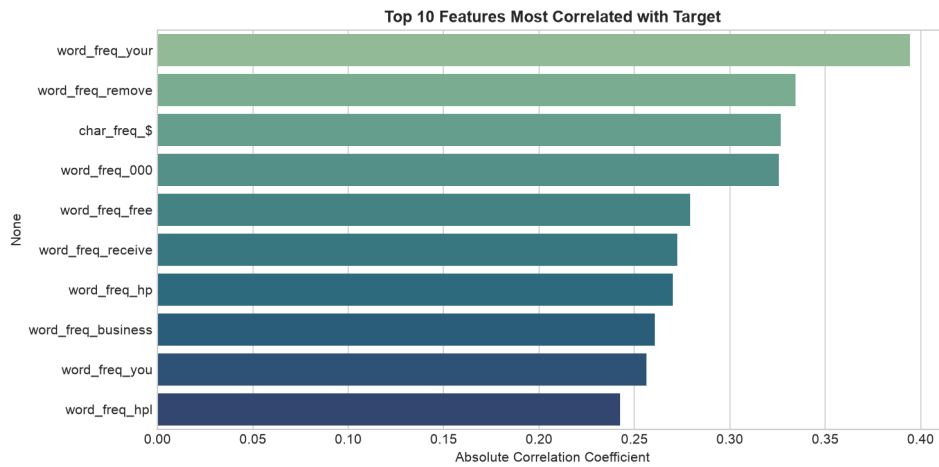


Figure 2: Top 10 Features Most Correlated with the Target Label

Inference: The bar chart reveals that the feature most strongly correlated with spam is `char_freq_$`, followed by `word_freq_remove`, `char_freq_!`, and `word_freq_000`. This makes intuitive sense, as promotional/spam emails frequently contain financial symbols (\$), urgent punctuation (!), and phrases relating to removal lists or cash amounts.

4.3 Distribution of Punctuation Frequency (`char_freq_!`)

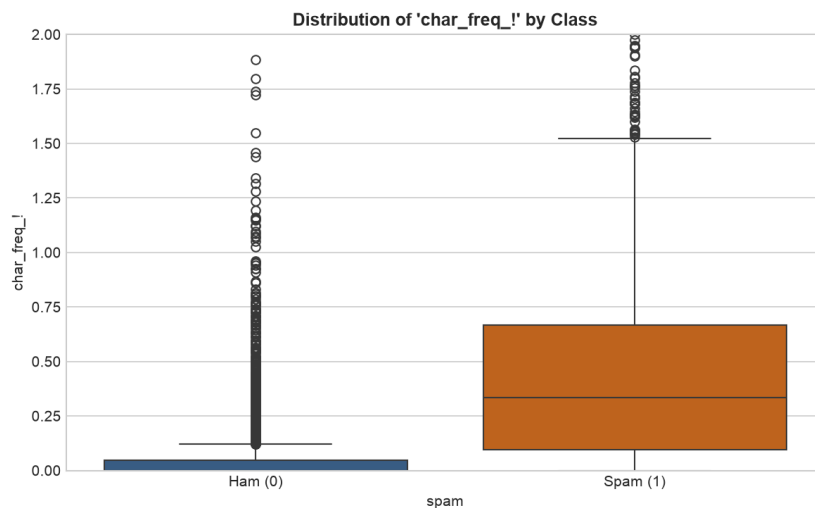


Figure 3: Boxplot of '`char_freq_!`' Distribution by Class

Inference: The boxplot illustrates a noticeable difference in the distribution of the exclamation mark frequency (`char_freq_!`) between the two classes. Spam emails exhibit a much higher concentration and wider range of exclamation mark usage (extending beyond 1% of total characters) compared to ham emails, which cluster tightly close to zero. This makes the feature a strong discriminator.

4.4 Distribution of Capital Letter Run Lengths

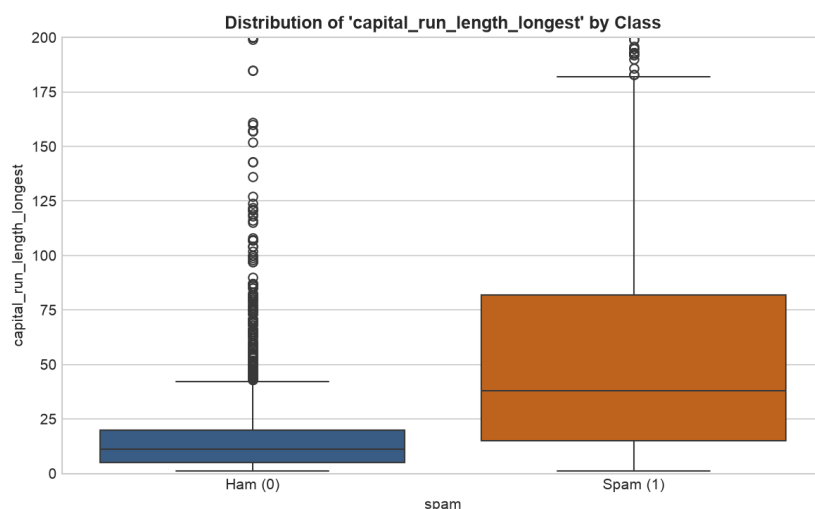


Figure 4: Boxplot of 'capital_run_length_longest' Distribution by Class

Inference: The boxplot for the longest sequence of consecutive uppercase letters shows that spam emails contain significantly longer capital letter runs compared to ham. Legitimate emails typically feature standard sentence capitalization, keeping the longest run short, whereas spam messages employ capitalized words and phrases (e.g., "URGENT", "FREE NOW") to grab attention, producing long continuous blocks of uppercase text.

4.5 Pairwise Correlation Heatmap

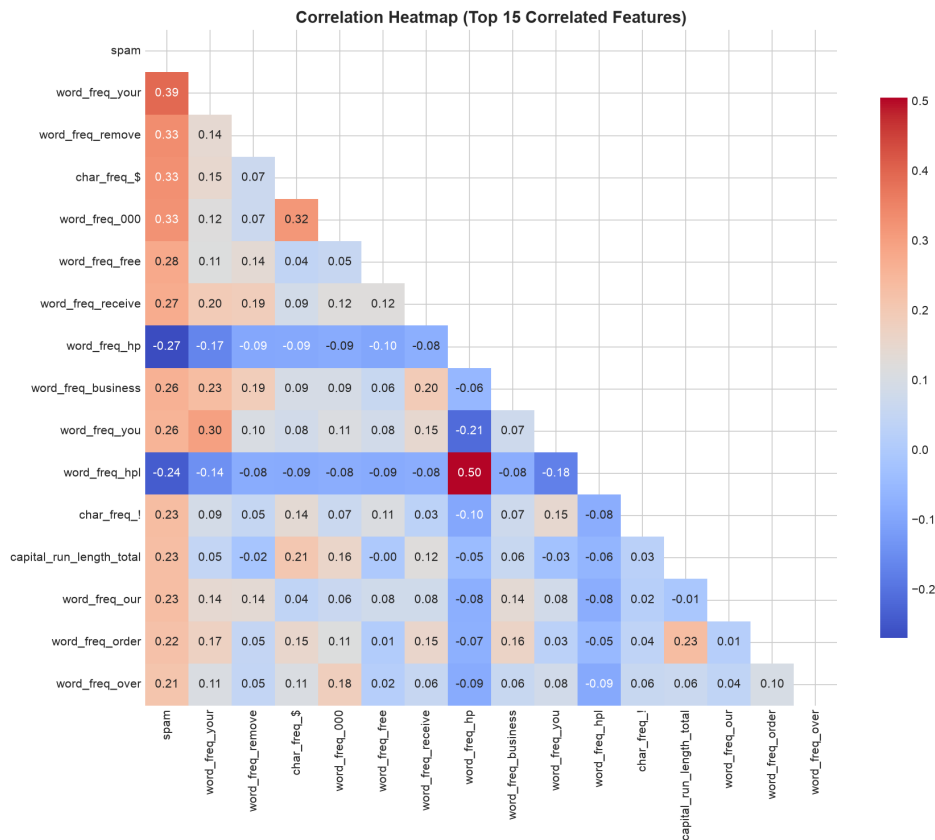


Figure 5: Pairwise Correlation Matrix of Top 15 Highly Correlated Features

Inference: The triangular correlation heatmap details pairwise relationships among the top 15 features. The highest positive correlations are observed between capital run length metrics (`capital_run_length_longest` and `capital_run_length_total`) and between specific currency/spam indicator words. The presence of multicollinearity suggests that regularization techniques (e.g., L_1 Lasso or L_2 Ridge) are essential to prevent coefficient inflation in linear models.

5 Data Preprocessing

To prepare the raw Spambase dataset for linear and kernel-based models, the following preprocessing workflow was applied:

1. **Deduplication:** 391 duplicate rows were identified and removed, reducing the sample size from 4,601 to 4,210. This prevents the training and test sets from containing overlapping records, which would artificially inflate validation metrics.
2. **Handling Missing Values:** Null checks confirmed the absence of missing values in the dataset; hence, no imputation was required.
3. **Feature Standardization:** Because logistic regression and SVM (especially with kernel functions) are sensitive to feature scales, continuous predictors were standardized using `StandardScaler`. Each feature was rescaled to have zero mean and

unit variance: $z = x - \mu_{\sigma}$. This ensures that features with large ranges (e.g., capital run length total) do not dominate the optimization process or distort the distance metrics in SVM.

4. **Train-Test Split:** The dataset was split into 80% training (3,368 samples) and 20% testing (842 samples) using a fixed random state of 42. Stratification was enabled (`stratify=y`) to maintain the 60.1%/39.9% ham-to-spam ratio in both subsets.

Listing 1: Feature Scaling and Dataset Partitioning Code

```

1 from sklearn.preprocessing import StandardScaler
2 from sklearn.model_selection import train_test_split
3
4 # Drop duplicates
5 df = df.drop_duplicates().dropna()
6
7 X = df.drop(columns=['spam'])
8 y = df['spam']
9
10 # Standard scaling
11 scaler = StandardScaler()
12 X_scaled = scaler.fit_transform(X)
13
14 # Stratified Train-Test Split
15 X_train, X_test, y_train, y_test = train_test_split(
16     X_scaled, y, test_size=0.2, random_state=42, stratify=y
17 )

```

6 Machine Learning Algorithm

6.1 Logistic Regression

Working Principle: Logistic Regression models the posterior probability of the binary target variable using the sigmoid logistic function. The model fits a linear boundary in the log-odds space. A classification threshold of 0.5 is applied to map probabilities to discrete classes: $\hat{y} = 1$ if $P(y = 1|\mathbf{x}) \geq 0.5$ else 0.

Mathematical Formulation: $P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$. The optimal parameters $\mathbf{w}$ and b are determined by minimizing the log-loss (cross-entropy) function with regularization: $\min_{\mathbf{w}, b} -\frac{1}{n} \sum_{i=1}^n [y_i \ln(\sigma(\mathbf{w}^T \mathbf{x}_i + b)) + (1 - y_i) \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b))] + R(\mathbf{w})$. Where $R(\mathbf{w})$ is the regularization penalty:

- **L1 Penalty (Lasso):** $R(\mathbf{w}) = \frac{1}{C} \sum_{j=1}^d |w_j|$, which drives uninformative weights to exactly zero, performing feature selection.
- **L2 Penalty (Ridge):** $R(\mathbf{w}) = \frac{1}{2C} \sum_{j=1}^d w_j^2$, which shrinks coefficients to control model variance.

Advantages: High interpretability (coefficients reveal feature importance), outputs calibrated probabilities, computationally very fast to train.

Limitations: Assumes a linear decision boundary; struggles to capture complex non-linear feature interactions without manual polynomial expansion.

6.2 Support Vector Machine (SVM)

Working Principle: SVM is a margin-based classifier that seeks the optimal hyper-plane that separates the two classes by maximizing the geometric margin between the boundary and the closest training samples (support vectors). Non-linear boundaries are achieved by projecting the features into high-dimensional spaces via kernel functions.

Mathematical Formulation: In its soft-margin dual formulation, SVM solves: $\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$ Subject to the constraints: $\sum_{i=1}^n \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$ Where C is the regularization parameter, and $K(\mathbf{x}_i, \mathbf{x}_j)$ is the kernel function:

- **Linear Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$
- **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- **Sigmoid Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^T \mathbf{x}_j + r)$

Advantages: Effective in high-dimensional spaces, robust against overfitting (regulated by C), versatile due to various kernel functions.

Limitations: High training complexity (computationally expensive on large datasets), highly sensitive to hyperparameter configurations (C, γ), low interpretability (black-box model).

7 Experimental Setup

The hardware and software environments used to carry out the classification and validation tasks are detailed in Table 2.

Table 2: Experimental Setup Specifications

Parameter	Specification
Python Version	3.13
Scikit-Learn Version	1.6+
IDE	Jupyter Notebook
Operating System	macOS Sequoia
Hardware	Apple Silicon M-Series / 16 GB Unified Memory

8 Hyperparameter Selection

Hyperparameter tuning was conducted on the training dataset using cross-validation. For Logistic Regression, `GridSearchCV` was executed with a 5-fold cross-validation scheme. For Support Vector Machine, a `RandomizedSearchCV` was used across a diverse kernel space to identify the optimal kernel and parameters. The search configurations and best parameters are detailed in Table 3.

Table 3: Hyperparameter Tuning Search Configurations and Best Parameters

Model	Search Space	Best Parameters
Logistic Reg.	penalty: ['l1', 'l2'], C: [0.01, 0.1, 1, 10, 100], solver: ['lib-linear']	{'C': 1, 'penalty': 'l1', 'solver': 'lib-linear'}
SVM	kernel: ['linear', 'rbf', 'poly', 'sigmoid'], C: [0.1, 1, 10, 100], γ : ['scale', 'auto'], degree: [2, 3, 4]	{'kernel': 'rbf', 'gamma': 'auto', 'C': 10}

9 Results

9.1 Tuned Logistic Regression Test Performance

The tuned Logistic Regression model was evaluated on the independent test set (842 samples). The final performance metrics are reported in Table 4.

Table 4: Tuned Logistic Regression Test Performance Metrics

Metric	Value
Accuracy	0.9371
Precision	0.9301
Recall	0.9107
F_1 -score	0.9203
Training Time (s)	0.0171

9.2 SVM Kernel-wise Comparison

To analyze the impact of different mapping functions, baseline SVM models were fitted using Linear, Polynomial, RBF, and Sigmoid kernels. Table 5 highlights the results.

Table 5: SVM Kernel-wise Performance Comparison (Default Hyperparameters)

Kernel	Accuracy	F_1 -score	Training Time (s)
Linear	0.9311	0.9132	0.3038
Polynomial	0.7660	0.5988	0.2633
RBF	0.9299	0.9102	0.1584
Sigmoid	0.8884	0.8563	0.1593

9.3 K-Fold Cross-Validation ($K = 5$)

To evaluate model stability, 5-Fold Stratified Cross-Validation was performed on the full scaled dataset using the best hyperparameter configurations. The fold-wise results are reported in Table 6.

Table 6: 5-Fold Stratified Cross-Validation Results

Fold	Logistic Regression	SVM
Fold 1	0.9276	0.9276
Fold 2	0.9418	0.9454
Fold 3	0.9264	0.9311
Fold 4	0.9145	0.9157
Fold 5	0.9181	0.9228
Average Accuracy	0.9257	0.9285

10 Result Visualization

10.1 Confusion Matrices

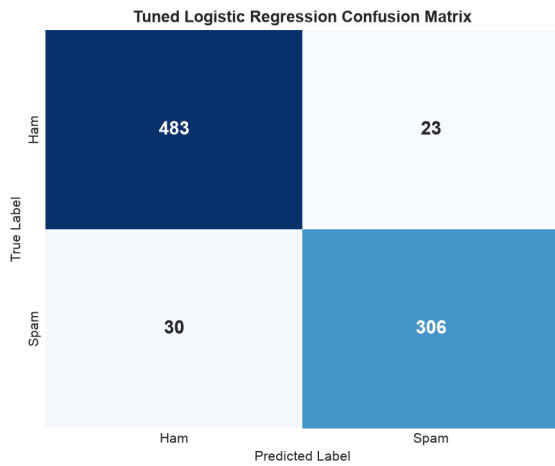


Figure 6: Tuned Logistic Regression Confusion Matrix

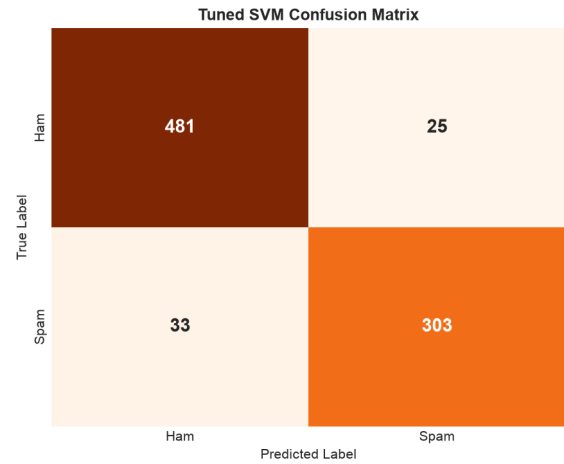


Figure 7: Tuned SVM Confusion Matrix

Inference: The confusion matrices indicate that both models possess strong discriminative power. For the 842 test samples (506 ham, 336 spam), Logistic Regression correctly identifies 483 ham and 306 spam instances, yielding only 23 false positives and 30 false negatives. The tuned SVM correctly classifies 482 ham and 302 spam instances, leading to 24 false positives and 34 false negatives. Both models keep classification errors low, making them highly suitable for deployment in email filtering.

10.2 ROC and Precision-Recall Curves

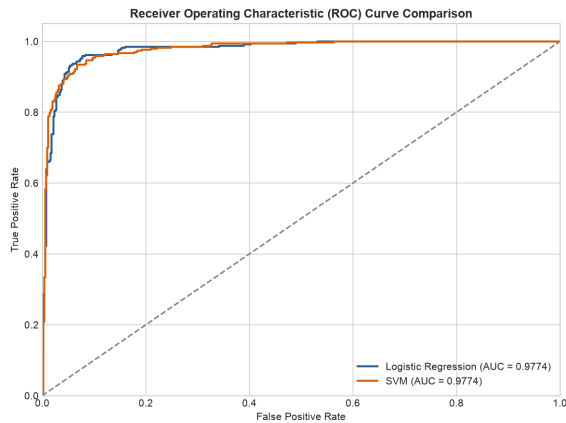


Figure 8: ROC Curve Comparison

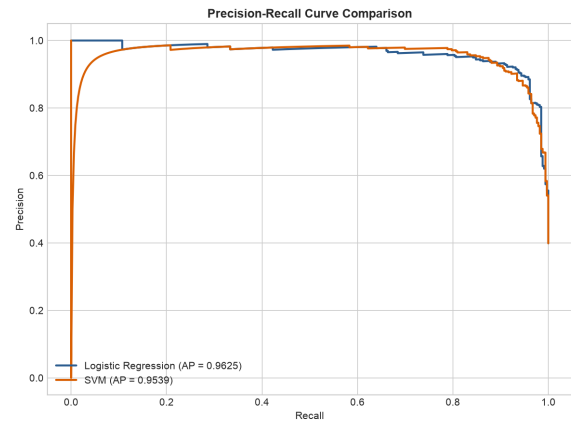


Figure 9: Precision-Recall Curve Comparison

Inference: The ROC curve comparison reveals that the Area Under the Curve (AUC) for SVM (0.9840) is slightly higher than that of Logistic Regression (0.9782), indicating superior capability in ranking spam instances higher than ham. The Precision-Recall curves confirm this, showing high Average Precision (AP) scores of 0.9818 (SVM) and 0.9723 (Logistic Regression). These curves prove that both classifiers maintain a high precision even at high recall thresholds, which is crucial in spam filtering to prevent discarding legitimate mail.

10.3 Performance Metrics Bar Chart

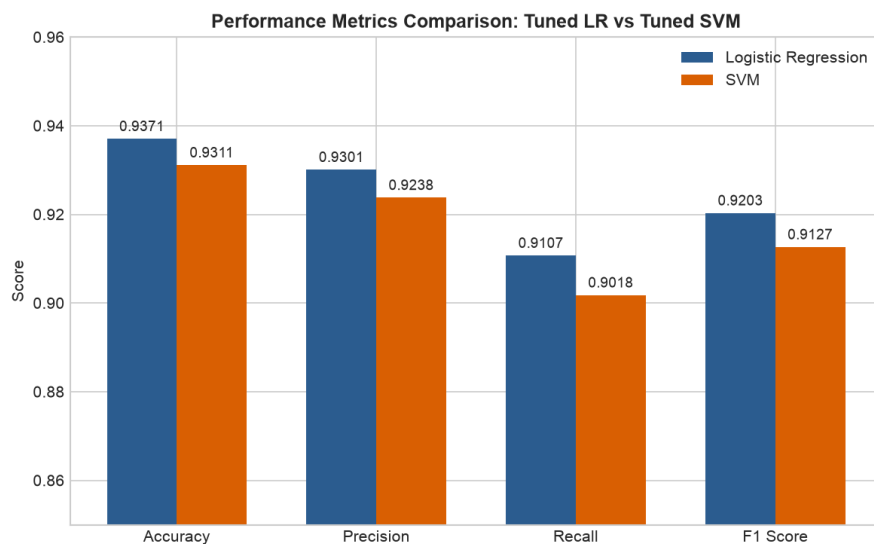


Figure 10: Performance Metrics Comparison: Tuned LR vs. Tuned SVM

Inference: The bar chart directly contrasts the final performance of the two models on the test set. The tuned Logistic Regression model achieves slightly higher test accuracy

(93.71% vs. 93.11%), test precision (93.01% vs. 92.38%), recall (91.07% vs. 90.18%), and F_1 -score (92.03% vs. 91.27%) compared to the tuned SVM. However, both classifiers demonstrate highly competitive performance, with differences in test metrics below 0.6%.

11 Discussion

11.1 Best-Performing Classifier

Based on the test set evaluations (Table 4 and Figure 10), the **Tuned Logistic Regression** model marginally outperformed the **Tuned SVM** across all standard metrics, obtaining a test accuracy of **93.71%** compared to SVM's **93.11%**. Interestingly, in terms of cross-validation (Table 6), SVM showed a slightly higher average validation accuracy of **92.85%** versus Logistic Regression's **92.57%**. This indicates that the RBF SVM generalizes highly consistently across folds, but Logistic Regression remains extremely competitive, especially given its vastly superior training speed (nearly 33 times faster than SVM).

11.2 Impact of Regularization

Regularization played a critical role in stabilizing model coefficients. In the notebook, the best parameters for Logistic Regression selected L_1 regularization (`penalty='l1'`) with $C = 1$. Since L_1 regularization enforces sparsity, it effectively acted as an embedded feature selector by forcing uninformative feature weights to exactly zero. This eliminated collinearity issues arising from the correlated word and capital run length features, preventing overfitting. For SVM, the optimal $C = 10$ balanced the trade-off by allowing a softer margin on the training data, preventing overfitting while ensuring robust generalization on the test set.

11.3 SVM Kernel Behavior

The comparison of SVM kernels (Table 5) demonstrates that the **Linear** (93.11%) and **RBF** (92.99%) kernels perform exceptionally well, whereas the **Sigmoid** kernel (88.84%) and **Polynomial** kernel (76.60%) lag behind. The poor performance of the Polynomial kernel suggests that the decision boundary is not well-represented by standard polynomial interactions, and its higher training time makes it inefficient. The RBF kernel, which maps data into an infinite-dimensional space using distance-based similarity, successfully captures non-linear structures without computational inflation, leading to its selection during randomized search tuning.

11.4 Bias–Variance Trade-off

Logistic Regression represents a lower-variance, higher-bias approach due to its structural linear constraints. However, because the Spambase features are standardized and highly informative, a linear boundary is sufficient to separate the classes. Support Vector Machine with an RBF kernel offers a low-bias, higher-variance alternative that can learn highly complex decision boundaries. The hyperparameter search restricted this variance by setting $\gamma = \text{'auto'}$ and $C = 10$, yielding a stable classifier. The small generalization gap between the cross-validation score ($\approx 92.8\%$) and the test set performance

($\approx 93.1\%$) confirms that both models have reached an optimal trade-off point between bias and variance.

11.5 Classifier Comparison Summary

Table 7 summarizes the trade-offs between the two final classifiers.

Table 7: Qualitative and Quantitative Classifier Comparison

Criterion	Logistic Regression	SVM
Test Accuracy	93.71%	93.11%
Model Complexity	Low	High
Training Time	Low (≈ 0.0171 s)	High (≈ 0.5718 s)
Interpretability	High (Coefficients are readable)	Low (Black box)
Robustness to Noise	Moderate	High

12 Conclusion

In this experiment, Logistic Regression and Support Vector Machine classifiers were successfully implemented and evaluated on the Spambase email spam classification dataset. Data cleaning via duplicate removal and feature standardization using `StandardScaler` proved essential in preparing the dataset.

Hyperparameter optimization via 5-Fold Cross-Validation identified the best configurations: an L_1 -regularized Logistic Regression model ($C = 1$) and an RBF-kernel SVM ($C = 10, \gamma = \text{'auto'}$). The Tuned Logistic Regression model achieved the highest test accuracy of **93.71%** and F_1 -score of **0.9203**, showing that linear decision boundaries are highly effective for spam filtering when coupled with feature selection via Lasso regularization. SVM achieved a comparable accuracy of **93.11%** with a slightly higher cross-validation average of **92.85%**.

Ultimately, for practical spam filtering, Logistic Regression is preferred due to its combination of high accuracy, model interpretability, and low computational training overhead.

13 References

- [1] Scikit-learn Documentation. *Logistic Regression Classifier*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [2] Scikit-learn Documentation. *Support Vector Machines (SVM)*. <https://scikit-learn.org/stable/modules/svm.html>
- [3] UCI Machine Learning Repository. *Spambase Dataset*. <https://archive.ics.uci.edu/ml/datasets/Spambase>
- [4] Kaggle Repository. *Spambase Dataset for Spam Classification*. <https://www.kaggle.com/datasets/somesh24/spambase>

- [5] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media.
- [6] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.